

PYTHON: basics

- знайомство з bool

bool: булева логіка

в 1854 році Джордж Буль в книзі “Закони мислення” описав “булеву алгебру” - яка спирається на два значення True (істина) та False (брехня) і визначає логічні операції - AND (“і”), OR (“або”), NOT (“ні”)

для роботи з такими даними в Python існує спеціальний тип даних - bool, який визначає всього два можливих об’єкта - True та False і набір операцій над ними - and, or, not

це дуже зручний і часто використовуваний в програмуванні тип даних, тому обов’язково треба вміти працювати з ним.

```
>>> a = True
>>> type(a)
>>> <class 'bool'>
>>> isinstance(a, bool)
>>> True
```

bool: логічні операції, оператор not

булева алгебра визначає два можливих значення для своїх операндів (**True** та **False**) і три можливі операції (**and**, **not**, **or**). При цьому - оператор **not** - одномісний (тобто як операція “-” в арифметиці - яка очікує одне число справа і змінює знак цього числа - якщо число додатне то поверта відповідне від’ємне, а якщо число від’ємне - повертає додатне). Оператор **not** “перевертає” значення операнда - якщо отримає **True** - поверне **False**, а якщо на вхід отримає **False** - поверне **True**:

```
>>> not True
>>> False
>>> not False
>>> True
```

для опису роботи логічних операторів часто використовують “таблиці істинності” - які демонструють що на вході і, відповідно, що на виході. Для оператора **not** це виглядає так:

Вхідні параметри	
a	not a
True	False
False	True

bool: оператор and

оператор **and** (“і”) повертає True лише коли всі вхідні операнди True. Ми часто несвідомо використовуємо таку логічну операцію в житті, наприклад: “ми поїдемо на море якщо будуть гроші і дадуть відпустку”. Тобто - обов’язкове виконання двох умов разом - “будуть гроші” і “дадуть відпустку”. Таблиця істинності виглядає так:

відповідно у python все реалізується доволі очікувано:

```
>>> True and True
>>> True
>>> False and True
>>> False
>>> False and False
>>> False
```

Вхідні параметри		a and b
a	b	
True	True	True
True	False	False
False	True	False
False	False	False

bool: оператор or

оператор **or** (“або”) повертає True коли хоча б один з вхідних операндів True і False якщо всі вхідні операнди False. Ми часто несвідомо використовуємо таку логічну операцію в житті, наприклад: “якщо виплатять премію або відпускні - одразу забронюємо базу відпочинку”. Тобто - виконання будь-якого з двох умов - “премію” або “відпускні” - гарантує позитивний результат. Таблиця істинності виглядає так:

відповідно у python все реалізується доволі очікувано:

```
>>> True or True
>>> True
>>> False or True
>>> True
>>> False or False
>>> False
```

Вхідні параметри		a or b
a	b	
True	True	True
True	False	True
False	True	True
False	False	False

bool: логічні вирази

часто є результатом роботи інших операторів або функцій

```
>>> a = 10
>>> b = 10
>>> c = 15
>>> a is b
True
>>> a > c
False
```

також ми можемо створювати більш складні логічні вирази, які складаються з кількох логічних операторів і операндів:

```
>>> a == b and b < c
True
```


bool: як перетворюються типи

дуже часто об'єкти одного типу можуть перетворюватись в інший тип.

Наприклад кожне число - як число - має відповідне зображення як рядок:

```
>>> num = 12
>>> num
12
```

зверніть увагу що воно зображено без лапок. Тому що це зараз - саме число:

```
>>> type(num)
<class 'int'>
```

але існує функція `str()` яка приймає об'єкт і перетворює його на відповідний рядок:

```
>>> str_num = str(num)
>>> str_num
'12'
>>> type(str_num)
<class 'str'>
```

зверніть увагу що тепер значення `str_num` відображається в лапках - тому що це рядок. Результат перевірки типу за допомогою функції `type()` це підтверджує.

bool: відповідність інших типів

Точно так же як у типа даних `str` є функція `str()`, яка перетворює об'єкти в їх рядкове представлення, так і у класа `bool` є функція `bool()` яка отримує об'єкт якогось класу і намагається перетворити його в об'єкт класу `bool` - а ми пам'ятаємо що там можливо лише два варіанта - `True` та `False`.

Правила перетворення для кожного класу свої і ми будемо говорити про них по ходу вивчення класів. Наприклад для рядків вони дуже прості - якщо рядок не пустий (тобто в ньому є хоч один який-небудь символ - навіть пробіл) - то це перетворюється у `True`, пусті рядки це завжди `False`

```
>>> bool("")  
False  
>>> bool(" ")  
True
```


bool: summary

- Python має вбудований тип даних bool, який представлений двома можливими значеннями: True, False
- цей тип даних використовується, наприклад, для побудови логічних виразів.
- для екземплярів цього типу реалізовані оператори які реалізують операції булевої алгебри: not, and, or
- ми познайомились з “конструкторами типів” (**str(obj)**, **bool(obj)**) які приймають об’єкт будь-якого типу і намагаються перетворити його за певними правилами в об’єкт свого типу (якщо це можливо).

Самостійне опрацювання:

- в REPL поекспериментуйте з перетворенням різноманітних об'єктів (числа, рядки, функції) в вивчені Вами типи даних за допомогою конструкторів типів (str, bool)
- Напиши скрипт який оцінює результат наступного вислову на основі базових параметрів і друкує результат: “наша команда вийде в наступний етап чемпіонату якщо ми переможемо команду “Метелик”, не програємо команді “Мураха”, а команда “Муха” або програє “Метелику”, або зіграє в нічию з командою “Мураха””.

Наступний матеріал: цілі числа і числа з плаваючою комою (int, float)